UNIVERSITY OF St.Thomas

SEIS 606: Vibe Coding

Fall 2026

Paul Kaefer, Adjunct Professor

Tonight's Agenda

1. Discuss Homework #1
2. Software Design & AI-Assisted Design
3. Test Cases & UI Mockups
4. Spec-Driven AI-Assisted Development
5. Tool setup/demo

Homework 1 discussion

Volunteers to share their repo?

Did everyone apply (and get approved) for Copilot?

Disclaimers

Claude was helpful for idea generation & helping organize tonight's lecture.

I borrow a lot from Justin Grammens' talk at Open Source North (GitHub). I cite this as "Grammens 2026" in tonight's slide deck.

2:47 AM. Room 412. Insulin overdue. The alert system shows no overdue alerts. The code looks clean. Tests pass.

Story from Grammens 2026

Software Design Principles

We talked about Test-Driven Development (TDD). There are also paradigms such as data-driven development and behavior-driven development.

separation of concerns (SoC): idea of taking a complex problem and breaking it into distinct aspects/issues

- Allows for modular development

Why design before we code?

List of software development philosophies on Wikipedia is a good "landing page" with more.

What does AI accelerate?

Ideation

- Quickly build architecture and *scaffolding* (code generation for fundamental structure of your application)
- Produce trade-off tables
- Analysis of potential flaws (can include "red teaming")

Documentation

- Generated from code & vice-versa

Testing

- Create a test suite & help identify gaps

What does AI *not* replace?

- Judgement about which trade-offs to choose
- Really any form of intent (*should* the software do this or that?)
- Long-term maintainability

Vibe coding vs. disciplined AI-assisted design

What are your thoughts?

"Vibe Coding"

- Buzz word for AI-assisted design?
- "Rick Rubin with headphones on building an Ableton plug in" – John
- Good way to involve folks who wouldn't otherwise know anything about JavaScript, etc.
- Feels like we're in "the podcast era"
- Feels like cowboys: anyone can do it

AI-assisted design

- John argues: it's human-assisted AI design
- More professional-sounding
- Sounds better on a resume
- Raphael: what about "prompt-led development"?/prompt engineering

Either way, good for people new to programming languages

THE CORE CONTRAST

Same AI. Completely Different Engineering.

✕ VIBE CODING PROMPT

```
"Build me a medication alert system that
notifies nurses when doses are late"
```

What gets generated:

- setTimeout-based timing (no UTC, no injection)
- Hardcoded email function, no role map
- No state machine — flag variable
- No audit log, no actor tracking

Vibe coding has a place: exploration, research, quick bug hunts.

✓ SPEC-DRIVEN PROMPT

```
"Implement DoseStateEngine satisfying REQ-001 through
REQ-012. Pure function. Injected timestamps. JSDoc
tracing each method to requirement ID."
```

What gets generated:

- ✓ Pure function — no hidden state
- ✓ Injected timestamps (deterministic, testable)
- ✓ Explicit state machine — all transitions typed
- ✓ Immutable audit entry on every transition
- ✓ JSDoc: @satisfies REQ-001 per method
- ✓ Tests labeled per requirement, boundary cases covered

Spec-driven is non-negotiable when traceability is required.

Source: Grammens 2026

Let's discuss

When in past projects did skipping design have a cost down the road?

- Vague prompting for personal website → more iterations required
  - “garbage in, garbage out”
- Patti’s consulting experience: similar to the above, if the focus is on getting it to work, other features may be lacking/issues may arise
- Corporations: because of budget, design lacks, causing a need for more expensive personnel to come in to fix
- “legacy system” that wasn’t designed well, but is “irreplaceable” because everyone uses it
  - Banking system, COBOL
- Building design, on paper → looked good, but didn’t have architectural experience. End result: sleeping area was smaller than optimal.



Test Cases & UI Mockups

First, an app idea for our class?

What's something we could build as a class?

My constraints:

- Ideally a web interface
- Something relatively small... let's not rebuild a social network or Salesforce
- Maybe something fun?

"Structure before Syntax"

- Mockups and tests are related: building a "contract" that work must follow
- Both can be (should be) documented
- Try with AI?

Example feature to mock-up

Our idea:

- App/website: upload a photo of food & it will estimate calories (and ???)
  - Full nutrition panel, including macro/micronutrients, saturated fat
- Food4Thought as name

We're "vibe coding" an app called Food4Thought where users can upload a photo of food & it will return a nutrition profile, including calories, vitamins/minerals, etc. We want it to be a web app. Generate a mock-up for the interface. We want it to have a "nutrition influencer" vibe.

Ideas:

- eventually: AR?
- part of the process: identify food items + portion size (volume? Weight?)
- Photo after & it will estimate how much you ate
- Any photo (plate, charcuterie board, etc.)

How did AI do?

What did it do well?

- Navigation buttons work
- Unified design, color palette, consistency
  - It seemed to follow what we looked for (probably lots of examples)

What did it get wrong?

- It (perhaps correctly) chose more of a social media route vs. a feel around us protecting health data, etc.

What did it think of that we didn’t?

- “community” aspect Paul hadn’t thought about
- Gut health, macros

What is not included (yet)

- Pricing!
- Let’s think about health data… any concerns?

Dedicated design tools still have benefits

Figma, Lucidchart, draw.io, others:

- Work well for collaboration & early-stage design
- Great if you are still brainstorming!
- Sequence/state diagrams that aren't necessarily easy to get right with AI (yet)



Can start with a sketch, then feed into AI.

OR can ideate with AI, then build/tweak a flow yourself.

Test cases from specs

Behavioral-Driven Development (BDD): Write Acceptance Criteria as Given/When/Then

- **Given** <some initial context>
- **When** <event that triggers a scenario>
- **Then** <expected outcome>

The more specific you get, the easier it will be to generate tests.

Let's brainstorm a few

Spend 10 minutes coming up with **three** test cases for your app idea

- **Given** <some initial context>
- **When** <event that triggers a scenario>
- **Then** <expected outcome>

Test cases from specs

See Test Structure in the Grammens repo:

Test Structure

Tests are organized by requirement — one `describe` block per REQ-NNN:

```
detect.test.ts
  describe('REQ-001: detect dose overdue past threshold')      – 6 tests
  describe('REQ-002: emit care team alert')                     – 11 tests
  describe('REQ-003: no duplicate alerts')                      – 3 tests
  describe('REQ-010: determinism')                              – 6 tests

config-validator.test.ts
  describe('REQ-011: alertThresholdMinutes validation')         – 14 tests
  describe('REQ-012: escalationThresholdMinutes validation')    – 14 tests
  describe('structural validation')                             – 6 tests
```



Spec-Driven, AI-Assisted Development

2:47 AM. Room 412. Insulin overdue.

The alert system shows no overdue alerts. The code looks clean. Tests pass.

But a developer, six weeks ago, copy-pasted a date comparison. Timezone handling is wrong for daylight saving time. There is no requirement that specified UTC. No test covering timezone edge cases. No audit log to surface the discrepancy.

Just a patient in need of overdue insulin.

The root cause wasn't the developer. It wasn't the timezone. It was the absence of structure.

No requirement captured the constraint. No test verified it. No trace connected requirement to code.

Source: Grammens 2026

Three pillars (based on Grammens' talk)

Specification-First Development — structure before syntax

Product-Minded Engineering — decisions trace to observable behavior, not implementation convenience

Disciplined Use of AI Tools — AI operates *within* constraints, not *instead of* them

Human-in-the-loop?

Specification-First Development — structure before syntax

- Humans write spec

Product-Minded Engineering — decisions trace to observable behavior, not implementation convenience

- Human-centric design

Disciplined Use of AI Tools — AI operates *within* constraints, not *instead of* them

- Humans evaluate AI operation

Now, for the tool setup

Which AI IDE should we use?

- GitHub Copilot – tonight's class and likely the next several
- Kiro – also free for students, built on/by AWS
  - John says it's really spec-driven (as is another tool called Kilo: https://kilo.ai/)
- Other platforms:
  - Claude Code (perhaps free for students for a year)
  - Cursor (potentially can apply for student discounts)
  - Windsurf (now Devin AI)
  - Replit
  - Antigravity
  - Lovable
  - N10 AI
  - Zed, pi.dev: options where you can switch between tools

What else have you used? Cline and Roo Code are good if you are in VS Code https://cline.bot/

You should install...

- VS Code & GitHub Copilot Chat

- On Windows, you might need to install Git for the command-line
  - winget install --id Git.Git -e --source winget

- In PowerShell on Windows or Terminal on Mac
  - If needed, install uv first
  - uv tool install specify-cli --from git+https://github.com/github/spec-kit.git

In the Open Source North demo, Claude Code was used

I left these steps in as an option; I will be using Copilot.

Install node package manager, then:

- nvm install 24
- nvm use 24
- npm install -g @anthropic-ai/claude-code

Creating a new repo using SpecKit

Create a repo/folder. Inside it, run the following:

```
specify init .
```

Select which AI tool (`copilot` today, but up to you!)

SpecKit demo

The SpecKit workflow, mapped to commands

/speckit-constitution → non-negotiable principles, safety constraints, governance

/speckit-specify → requirements (REQ-NNN), acceptance scenarios, boundary conditions

/speckit-plan → architecture, constitution check, traceability map, ADRs

/speckit-tasks → task breakdown, file paths, REQ-NNN per task

The spec prompt format

OBJECTIVE — the failure mode, not the feature description

BEHAVIOR — observable outcomes only, no tech details

CONSTRAINTS — non-negotiables regardless of implementation

VERIFICATION — testable criteria, not subjective ones

Start with our Constitution

Like a country's constitution, this governs how everything goes (in theory!)

Example from this tutorial:

/speckit.constitution Create principles focused on code quality, testing standards, user experience consistency, and performance requirements.

This is for a web application where users will upload photos. Nutritional information will be provided back. We need to be concerned with quality of photos, and edge cases such as non-food items in photo.

Next is the specify phase

Using our app idea, let's follow the example at https://roelantd.github.io/spec-kit-workshop/03-specify-phase

Closing concepts

Again, borrowing from Grammens 2026:

1. AI doesn't make bad engineering good. It makes bad engineering faster.

2. A spec is not bureaucracy. It's the difference between building fast and building **right** fast.

3. Traceability isn't a compliance artifact. It's how you prove — to yourself, your team, and stake holders — that you knew what you were doing.



Homework #2

1. Decide on an app idea
2. Develop preliminary constitution and specs yourself
3. Storyboard/UI mockup

UNIVERSITY OF St.Thomas

Any questions before we end?